SIMULADOR DE ROBOT EN CUADRÍCULA 2D

Estudiante: Fernando André Chavarria Alfaro
Código: 74032240
Profesor: Mg. Gervasio Coronel Molina
Fecha: 04 de diciembre del 2025

ÍNDICE

1. Introducción
2. Enunciado del Ejercicio
3. Análisis del Ejercicio
4. Explicación de las Técnicas Usadas
5. Código de los Programas
6. Conclusiones
7. Apéndice

1. INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de un simulador de robot que se desplaza en una cuadrícula bidimensional. El objetivo principal es aplicar los conceptos de Programación Orientada a Objetos (POO) junto con el desarrollo de interfaces gráficas mediante la librería Tkinter de Python.

El simulador permite al usuario crear un robot en una posición inicial específica, elegir su dirección (norte, sur, este u oeste), y ejecutar una serie de instrucciones para moverlo dentro de un área delimitada. El robot deja un rastro visual de su recorrido y valida que no salga de los límites establecidos.

Este proyecto integra conceptos fundamentales de programación como clases, métodos, manejo de eventos, validación de datos y animaciones, demostrando la capacidad de diseñar soluciones modulares y escalables.

2. ENUNCIADO DEL EJERCICIO

Desarrollar un simulador de robot utilizando Programación Orientada a Objetos que cumpla con las siguientes características:

Requisitos funcionales:

- El robot debe moverse en una cuadrícula 2D con coordenadas (x, y)
- El robot tiene cuatro direcciones posibles: norte, sur, este y oeste
- El robot acepta tres tipos de instrucciones:
    - R: girar a la derecha (90 grados en sentido horario)
    - L: girar a la izquierda (90 grados en sentido antihorario)
    - A: avanzar una casilla en la dirección actual
- El sistema debe incluir una interfaz gráfica que permita:
    - Establecer la posición inicial del robot
    - Seleccionar la dirección inicial
    - Ingresar y ejecutar secuencias de instrucciones
    - Visualizar el movimiento del robot de forma animada
    - Mostrar el recorrido mediante líneas
- El robot no debe salir de un área delimitada de -8 a 8 en ambos ejes, en el enunciado elegido indica un plano infinito, sin embargo se usará delimitaciones
- El sistema debe validar las instrucciones antes de ejecutarlas

3. ANÁLISIS DEL EJERCICIO

3.1 Identificación de Componentes

El problema se puede descomponer en tres componentes principales:

Componente 1: Modelo del Robot (robot.py)

- Representa la lógica del robot independiente de la interfaz
- Maneja la posición actual (x, y)
- Maneja la dirección actual
- Implementa los métodos de movimiento y giro
- Mantiene un historial de posiciones visitadas

Componente 2: Interfaz Gráfica (interfaz_robot.py)

- Gestiona toda la interacción con el usuario
- Dibuja la cuadrícula y el robot
- Valida las entradas del usuario
- Anima el movimiento del robot
- Comunica los eventos del usuario al modelo

Componente 3: Programa Principal (main.py)

- Punto de entrada de la aplicación
- Instancia y ejecuta la interfaz

3.2 Flujo de Ejecución

1. El usuario inicia el programa
2. Se muestra la interfaz con la cuadrícula vacía
3. El usuario ingresa posición inicial (x, y) y dirección
4. Al presionar "Crear Robot", se instancia un objeto Robot y se dibuja en el canvas
5. El usuario ingresa una cadena de instrucciones (ej: "RAALARA")
6. Al presionar "Ejecutar", el sistema:
    - Valida que las instrucciones sean válidas (solo R, L, A)
    - Simula el recorrido para verificar que no salga de límites
    - Si es válido, ejecuta cada instrucción con una animación de 500ms
7. El usuario puede limpiar y reiniciar el robot

3.3 Arquitectura de la Solución

Se utilizó el patrón de diseño Modelo-Vista-Controlador (MVC):

- **Modelo:** Clase Robot (lógica del negocio)
- **Vista:** Canvas de Tkinter (visualización)
- **Controlador:** Clase InterfazRobot (gestiona eventos y coordina modelo-vista)

Esta separación permite que el modelo Robot sea reutilizable en otros contextos sin depender de la interfaz gráfica.

4. EXPLICACIÓN DE LAS TÉCNICAS USADAS

4.1 Programación Orientada a Objetos

Se utilizó POO para encapsular la lógica del robot en una clase independiente. Los principios aplicados fueron:

Encapsulación: Los atributos del robot (x, y, direccion) se mantienen dentro de la clase y se accede a ellos mediante métodos públicos (obtener_posicion, obtener_direccion).

Abstracción: La clase Robot expone una interfaz simple (girar_derecha, girar_izquierda, avanzar) ocultando la complejidad interna de cómo se calculan las nuevas posiciones.

Modularidad: Separar el modelo del robot de la interfaz permite modificar uno sin afectar al otro.

4.2 Sistema de Coordenadas

Se implementó una conversión entre dos sistemas de coordenadas:

- **Coordenadas del robot:** Sistema cartesiano tradicional donde Y crece hacia arriba
- **Coordenadas del canvas:** Sistema donde Y crece hacia abajo (típico en interfaces gráficas)

La conversión se realiza mediante:

```
x_canvas = centro_x + (x_robot * escala)
```

```
y_canvas = centro_y - (y_robot * escala)
```

4.3 Manejo de Direcciones con Lista Circular

Las cuatro direcciones se almacenan en una lista ordenada: ["norte", "este", "sur", "oeste"]. Para girar a la derecha se suma 1 al índice, y para girar a la izquierda se resta 1, usando el operador módulo (%) para crear un comportamiento circular.

4.4 Validación Preventiva

Antes de ejecutar las instrucciones, se crea un robot temporal que simula todo el recorrido. Si en algún momento el robot temporal se sale de los límites, se rechaza la secuencia completa y se solicita al usuario ingresar nuevas instrucciones.

4.5 Animación con Recursión y Callbacks

La animación se implementó usando el método after() de Tkinter, que programa la ejecución de una función después de un tiempo determinado. El método animar_paso() se llama a sí mismo recursivamente hasta completar todas las instrucciones, creando el efecto de animación paso a paso.

4.6 Gestión de Estado

Se utilizan variables de estado como esta_animando para evitar que el usuario ejecute múltiples animaciones simultáneamente, lo que podría causar comportamientos inesperados.

5. CÓDIGO DE LOS PROGRAMAS

5.1 robot.py

```
#Utilizando POO para modelar un robot que se mueve en una cuadrícula 2D



class Robot:



    #Se incializa el robot en una posición y una dirección dada por el usuario

    def __init__(self, x, y, direccion):

        self.x = x

        self.y = y
```

```
        self.direccion = direccion
        self.historial = [(x, y)]
        self.direcciones = ["norte", "este", "sur", "oeste"]

    #Agregamos metodos para girar y mover al robot
    def girar_derecha(self):
        indice = self.direcciones.index(self.direccion)
        self.direccion = self.direcciones[(indice + 1) % 4]

    def girar_izquierda(self):
        indice = self.direcciones.index(self.direccion)
        self.direccion = self.direcciones[(indice - 1) % 4]

    def avanzar(self):
        if self.direccion == "norte":
            self.y += 1
        elif self.direccion == "sur":
            self.y -= 1
```

```
        elif self.direccion == "este":

            self.x += 1

        elif self.direccion == "oeste":

            self.x -= 1



        self.historial.append((self.x, self.y)) #Guardamos las posiciones en un
historial



    #Uso una función para obtener la posición y dirección actual del robot

    def obtener_posicion(self):

        return (self.x, self.y)



    def obtener_direccion(self):

        return self.direccion



    def obtener_historial(self):

        return self.historial
```

```
    def reiniciar(self, x, y, direccion):

        self.x = x

        self.y = y

        self.direccion = direccion

        self.historial = [(x, y)]
```

5.2 interfaz_robot.py

```
#importo las librerias necesarias y mi libreria de robot

import tkinter as tk

from tkinter import messagebox

from robot import Robot



#creo el interfaz

class InterfazRobot:



    def __init__(self):

        #creo la ventana principal

        self.ventana = tk.Tk()

        self.ventana.title("Simulador de Robot")
```

```
self.ventana.geometry("580x650")



self.robot = None  #se incia sin robot

self.escala = 20  #le pongo 20 pixeles por casilla



#establezco los limites del canvas

self.ancho_canvas = 400

self.alto_canvas = 400

self.centro_x = self.ancho_canvas // 2

self.centro_y = self.alto_canvas // 2



self.instrucciones_cola = []

self.esta_animando = False

self.id_triangulo = None

self.ids_lineas = []

self.direccion_actual = "norte"

self.botones_dir = {}
```

```
        #ejecución

        self.crear_interfaz()

        self.dibujar_cuadricula()



    #método para crear los widgets

    def crear_interfaz(self):

        #Frame superior

        frame_sup = tk.Frame(self.ventana, bg="lightgray", pady=8)

        frame_sup.pack(fill=tk.X)



        tk.Label(frame_sup, text="Posicion Inicial:", bg="lightgreen",
font=("Arial", 10, "bold")).pack()



        frame_pos = tk.Frame(frame_sup, bg="lightgray")  #frame para "x" e "y"

        frame_pos.pack(pady=3)



        #etiquetas y entradas para "x" e "y"

        tk.Label(frame_pos, text="X:", bg="lightgray").pack(side=tk.LEFT,
padx=3)
```

```
        self.entrada_x = tk.Entry(frame_pos, width=6)

        self.entrada_x.pack(side=tk.LEFT, padx=3)

        self.entrada_x.insert(0, "0")



        tk.Label(frame_pos, text="Y:", bg="lightgray").pack(side=tk.LEFT,
padx=3)

        self.entrada_y = tk.Entry(frame_pos, width=6)

        self.entrada_y.pack(side=tk.LEFT, padx=3)

        self.entrada_y.insert(0, "0")



        tk.Label(frame_sup, text="Direccion:", bg="lightgray", font=("Arial",
10, "bold")).pack(pady=(5,3))



        frame_dir = tk.Frame(frame_sup, bg="lightgray")  #frame para los
botones de dirección

        frame_dir.pack()



        #creo los 4 botones de dirección

        for dir_nombre in ["norte", "sur", "este", "oeste"]:

            #defino a norte como inicial
```

```
if dir_nombre == "norte":
    color_inicial = "lightblue"
    relieve_inicial = tk.SUNKEN
else:
    color_inicial = "SystemButtonFace"
    relieve_inicial = tk.RAISED


btn = tk.Button(
    frame_dir,
    text=dir_nombre.capitalize(),
    width=8,
    bg=color_inicial,
    relief=relieve_inicial,
    command=lambda d=dir_nombre: self.seleccionar_direccion(d)
)
btn.pack(side=tk.LEFT, padx=2)
self.botones_dir[dir_nombre] = btn
```

```
        tk.Button(frame_sup, text="Crear Robot", bg="green", fg="white",
width=12, command=self.crear_robot).pack(pady=5)



        #Frame de instrucciones

        frame_inst = tk.Frame(self.ventana, bg="lightblue", pady=8)

        frame_inst.pack(fill=tk.X)



        tk.Label(frame_inst, text="Instrucciones (R, L, A):",
bg="lightblue").pack()

        self.entrada_inst = tk.Entry(frame_inst, width=35)

        self.entrada_inst.pack(pady=3)



        frame_botones = tk.Frame(frame_inst, bg="lightblue")  #frame para
ejecutar y limpiar

        frame_botones.pack()



        tk.Button(frame_botones, text="Ejecutar", width=10, bg="blue",
fg="white", command=self.ejecutar_instrucciones).pack(side=tk.LEFT, padx=3)

        tk.Button(frame_botones, text="Limpiar", width=10, bg="orange",
command=self.limpiar_todo).pack(side=tk.LEFT, padx=3)
```

```
#creo el canvas donde voy a dibujar
self.canvas = tk.Canvas(
    self.ventana,
    width=self.ancho_canvas,
    height=self.alto_canvas,
    bg="white",
    highlightthickness=1,
    highlightbackground="black"
)
self.canvas.pack(pady=8)


#Frame inferior para mostrar la posición
frame_inf = tk.Frame(self.ventana, bg="lightgreen", pady=6)
frame_inf.pack(fill=tk.X)
self.etiqueta_estado = tk.Label(
    frame_inf,
    text="Posicion: -- | Direccion: --",
    bg="lightgreen",
```

```
        font=("Arial", 9)

    )

    self.etiqueta_estado.pack()



#método para seleccionar dirección inicial

def seleccionar_direccion(self, direccion):

    self.direccion_actual = direccion



    #actualizo todos los botones

    for nombre, boton in self.botones_dir.items():

        if nombre == direccion:

            boton.config(bg="lightblue", relief=tk.SUNKEN)

        else:

            boton.config(bg="SystemButtonFace", relief=tk.RAISED)



#método que dibuja la cuadrícula

def dibujar_cuadricula(self):

    for i in range(-10, 11):
```

```
            x = self.centro_x + (i * self.escala)

            self.canvas.create_line(x, 0, x, self.alto_canvas,
fill="lightgray")




            y = self.centro_y - (i * self.escala)

            self.canvas.create_line(0, y, self.ancho_canvas, y,
fill="lightgray")




        #dibujo los ejes principales más gruesos

        self.canvas.create_line(self.centro_x, 0, self.centro_x,
self.alto_canvas, fill="gray", width=2)

        self.canvas.create_line(0, self.centro_y, self.ancho_canvas,
self.centro_y, fill="gray", width=2)




    #convierte coordenadas del robot a coordenadas del canvas

    def convertir_coordenadas(self, x_robot, y_robot):

        x_canvas = self.centro_x + (x_robot * self.escala)

        y_canvas = self.centro_y - (y_robot * self.escala)  #invertido porque
en canvas Y crece hacia abajo

        return x_canvas, y_canvas
```

```
#calcula los puntos del triángulo según la dirección

def obtener_puntos_triangulo(self, x_canvas, y_canvas, direccion):

    tam = 10



    if direccion == "norte":

        return [

            x_canvas, y_canvas - tam,

            x_canvas - tam//2, y_canvas + tam//2,

            x_canvas + tam//2, y_canvas + tam//2

        ]

    elif direccion == "sur":

        return [

            x_canvas, y_canvas + tam,

            x_canvas - tam//2, y_canvas - tam//2,

            x_canvas + tam//2, y_canvas - tam//2

        ]

    elif direccion == "este":

        return [
```

```
            x_canvas + tam, y_canvas,
            x_canvas - tam//2, y_canvas - tam//2,
            x_canvas - tam//2, y_canvas + tam//2
        ]
    else:  #oeste
        return [
            x_canvas - tam, y_canvas,
            x_canvas + tam//2, y_canvas - tam//2,
            x_canvas + tam//2, y_canvas + tam//2
        ]


#dibuja el robot en el canvas
def dibujar_robot(self):
    if self.id_triangulo:
        self.canvas.delete(self.id_triangulo)  #borro el triángulo anterior

    x, y = self.robot.obtener_posicion()
    direccion = self.robot.obtener_direccion()
```

```
        x_canvas, y_canvas = self.convertir_coordenadas(x, y)

        puntos = self.obtener_puntos_triangulo(x_canvas, y_canvas, direccion)



        self.id_triangulo = self.canvas.create_polygon(puntos, fill="red",
outline="black", width=2)



    #dibuja una línea entre dos puntos

    def dibujar_linea(self, x1, y1, x2, y2):

        x1_c, y1_c = self.convertir_coordenadas(x1, y1)

        x2_c, y2_c = self.convertir_coordenadas(x2, y2)

        linea = self.canvas.create_line(x1_c, y1_c, x2_c, y2_c, fill="blue",
width=2)

        self.ids_lineas.append(linea)



    #actualiza el texto de estado

    def actualizar_estado(self):

        if self.robot:

            x, y = self.robot.obtener_posicion()

            dir = self.robot.obtener_direccion()
```

```
            self.etiqueta_estado.config(text=f"Posicion: ({x}, {y}) |
Direccion: {dir}")



    #crea el robot con la posición y dirección inicial

    def crear_robot(self):

        try:

            x = int(self.entrada_x.get())

            y = int(self.entrada_y.get())

            self.robot = Robot(x, y, self.direccion_actual)



            #limpio el canvas y redibujo todo

            self.canvas.delete("all")

            self.dibujar_cuadricula()

            self.dibujar_robot()

            self.actualizar_estado()

            self.entrada_inst.delete(0, tk.END)

            self.ids_lineas = []
```

```
            messagebox.showinfo("Exito", f"Robot creado en ({x}, {y})")
        except ValueError:
            messagebox.showerror("Error", "X e Y deben ser numeros")


#verifica que el robot no se salga del área
def validar_limites(self, instrucciones):
    robot_temp = Robot(self.robot.x, self.robot.y, self.robot.direccion)
    x_min, x_max = -8, 8
    y_min, y_max = -8, 8

    #simulo todas las instrucciones
    for char in instrucciones:
        if char == 'R':
            robot_temp.girar_derecha()
        elif char == 'L':
            robot_temp.girar_izquierda()
        elif char == 'A':
            robot_temp.avanzar()
```

```
            x, y = robot_temp.obtener_posicion()
            if x < x_min or x > x_max or y < y_min or y > y_max:
                return False

        return True

#ejecuta las instrucciones ingresadas
def ejecutar_instrucciones(self):
    if not self.robot:
        messagebox.showwarning("Advertencia", "Primero crea un robot")
        return

    if self.esta_animando:
        return

    cadena = self.entrada_inst.get().upper().strip()
    if not cadena:
```

```
            messagebox.showwarning("Advertencia", "Ingresa instrucciones")
            return

        #valido que solo tenga R, L, A
        for char in cadena:
            if char not in ['R', 'L', 'A']:
                messagebox.showerror("Error", f"Caracter invalido: {char}")
                return

        #valido que no se salga de los límites
        if not self.validar_limites(cadena):
            messagebox.showerror("Fuera de limites", "El robot saldria del
area.\nIngresa nuevas instrucciones.")
            self.entrada_inst.delete(0, tk.END)
            return

        #inicio la animación
        self.instrucciones_cola = list(cadena)
```

```
    self.esta_animando = True
    self.animar_paso()

#anima cada paso de las instrucciones
def animar_paso(self):
    if not self.instrucciones_cola:
        self.esta_animando = False
        self.actualizar_estado()
        messagebox.showinfo("Completado", "Instrucciones ejecutadas")
        return

    inst = self.instrucciones_cola.pop(0)
    x_ant, y_ant = self.robot.obtener_posicion()

    if inst == 'R':
        self.robot.girar_derecha()
    elif inst == 'L':
        self.robot.girar_izquierda()
```

```
    elif inst == 'A':
        self.robot.avanzar()
        x_nueva, y_nueva = self.robot.obtener_posicion()
        self.dibujar_linea(x_ant, y_ant, x_nueva, y_nueva)

    self.dibujar_robot()
    self.actualizar_estado()
    self.ventana.after(500, self.animar_paso)  #espero medio segundo y sigo

#limpia todo y reinicia el robot
def limpiar_todo(self):
    if self.robot:
        x = int(self.entrada_x.get())
        y = int(self.entrada_y.get())
        self.robot.reiniciar(x, y, self.direccion_actual)

        self.canvas.delete("all")
        self.dibujar_cuadricula()
```

```
            self.dibujar_robot()

            self.actualizar_estado()

            self.entrada_inst.delete(0, tk.END)

            self.ids_lineas = []



    #inicia la aplicación

    def iniciar(self):

        self.ventana.mainloop()
```

5.3 main.py

```
from interfaz_robot import InterfazRobot



if __name__ == "__main__":  # inicia el programa

    app = InterfazRobot()

    app.iniciar()
```

6. CONCLUSIONES

Durante el desarrollo de este proyecto pude poner en práctica varios conceptos importantes de programación que había aprendido en clase. Las principales cosas que aprendí fueron:

1. **Separar el código en diferentes archivos:** Al principio pensé que sería más complicado tener tres archivos separados, pero en realidad hizo todo más fácil. Tener la lógica del robot en un archivo aparte me permitió probar el robot sin necesidad de abrir la interfaz gráfica cada vez. Además, si en el futuro quiero usar la clase Robot para otra cosa, puedo hacerlo sin problemas.
2. **Validar antes de ejecutar:** Una de las cosas más útiles que implementé fue crear un robot temporal que simula todo el recorrido antes de ejecutarlo. Esto evita que el usuario vea cómo el robot se sale del área y luego se muestre un error. Me parece mejor prevenir el error desde el inicio.
3. **El tema de las coordenadas fue complicado:** Me costó un poco entender que en el canvas el eje Y está invertido (crece hacia abajo en vez de hacia arriba). Pero una vez que hice la función de conversión, todo fue más simple. Creo que separar esta lógica en su propia función fue una buena decisión.
4. **Las animaciones mejoran mucho la experiencia:** Al principio ejecutaba todas las instrucciones de golpe, pero cuando agregué la animación paso a paso con el método after() el simulador se volvió mucho más interesante y fácil de entender. Ver cómo el robot se mueve paso a paso hace que sea más intuitivo.
5. **La lista circular para las direcciones:** Usar una lista con las cuatro direcciones y el operador módulo (%) para girar fue bastante ingenioso. Es mucho más simple que tener un montón de if/elif para cada caso de rotación.
6. **Importancia de validar todo:** Aprendí que siempre hay que validar lo que el usuario ingresa. En mi código reviso si el robot existe antes de ejecutar instrucciones, si las entradas X e Y son números válidos, y si las instrucciones solo contienen R, L y A. Esto hace que el programa sea más robusto.

En general, el proyecto me ayudó a entender mejor cómo funciona la Programación Orientada a Objetos en la práctica. Ver cómo la clase Robot puede existir independientemente de la interfaz gráfica me hizo entender mejor el concepto de encapsulación y modularidad. También me di cuenta de que organizar bien el código desde el inicio hace que sea más fácil agregar nuevas funcionalidades después.

7. APÉNDICE

7.1 Requisitos del Sistema

- Python 3.6 o superior
- Librería Tkinter (incluida por defecto en Python)
- Sistema operativo: Windows, macOS o Linux

7.2 Casos de Prueba Sugeridos

- Robot creado con posiciones negativas

- Giro a la derecha - Giro a la izquierda

- Avanzar una posición positiva hacia el norte

- Avanzar una posición negativa hacia el sur

- Avanzar una posición positiva hacia el este

- Avanzar una posición negativa hacia el oeste

- Movimiento oeste-norte

- Movimiento este-sur

7.3 Referencias

- Documentación oficial de Python: https://docs.python.org/3/
- Documentación de Tkinter: https://docs.python.org/3/library/tkinter.html
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- Matthes, E. (2019). Python Crash Course (2nd ed.). No Starch Press.